

Systèmes transverses — application automatique

Contrepartie *doctrine* du **WORKFLOW-daily.md** (qui, lui, est la checklist).
Ce fichier décrit les sept systèmes qui ne sont pas des étapes mais des **couches de gouvernance** : ils s'activent seuls, tout au long du cycle, sur déclencheur. Le diagramme **Systemes-transverses.png** en donne la carte mentale ; ce fichier en donne la règle.

Chaque fiche : **Déclencheur** (quand ça s'active) · **Application** (où, sur quel cycle) · **Effet automatique** (ce que ça impose sans y penser) · **Artefact** (le fichier/script qui l'incarne).

Maturité : **[OPÉRATIONNEL]** = prêt à l'usage · **[DIFFÉRÉ]** = activer quand ça fait mal (doctrine anti-préemption). On ne construit pas un système différé avant que son absence coûte.

1. Blast radius **[OPÉRATIONNEL]**

Le cadran d'entrée. Se règle **avant** toute modification et détermine le niveau de cérémonie de tout le reste.

- **Déclencheur** : au cadrage de chaque tâche, avant d'écrire une ligne. L'agent remplit un template d'analyse d'impact.
- **Application** : point d'entrée de la boucle courte (Day-cycle). Paramètre en cascade les quality gates, le contrat d'autonomie et le tiering.
- **Effet automatique** :
 - **Local** (1-2 fichiers, pas d'impact externe) → process léger : lint + tests unitaires. Autonomie haute.
 - **Feature** (multi-fichiers dans un module, endpoint, règle métier) → spec légère + TDD + quality gates complètes.
 - **Architecture** (multi-modules, DB, sécurité) → cycle documentaire complet
 - **validation humaine obligatoire**. Autonomie minimale.
- **Artefact** : template d'impact (dans **context/**), étape imposée par **AGENTS.md**, **quality-check.sh** paramétré par niveau. Rapport d'impact stocké dans le contexte (CDLC).

Point de vigilance : le blast radius **n'est pas contenu** dans les quality gates — il les *règle*. Il est en amont, pas en profondeur.

2. Quality gates [OPÉRATIONNEL]

La barrière déterministe. Exécutée par l'agent lui-même, jamais déléguée à une review manuelle submergée.

- **Déclencheur** : après chaque modification (boucle courte) et à chaque sortie de phase (cycle long).
- **Application** : deux points. Micro — validation-check de la boucle courte. Méso — gate de sortie de chaque phase SDLC. (Macro — merge distant conditionné au vert, côté Git.)
- **Effet automatique** : l'agent exécute **quality-check.sh** et corrige jusqu'à **100 %** avant toute soumission. TDD : tests rouges d'abord, code au vert ensuite. Un échec de gate déclenche une MAJ du contexte (CDLC Observe→Generate).
- **Artefact** : **scripts/quality-check.sh** (ruff/black/eslint · mypy/pyright · bandit/semgrep · pytest --cov-fail-under=80 · behave si Gherkin). Comportement imposé dans **AGENTS.md**.

3. Contrat d'autonomie [OPÉRATIONNEL]

La frontière homme-machine. Définit ce que l'agent fait seul et où il doit s'arrêter — le cœur de sécurité de la méthode.

- **Déclencheur** : en continu pendant l'exécution ; se manifeste aux points d'arrêt et au garde-fou de tentatives.
- **Application** : point « issue ? » de la boucle courte, et toute décision du cycle long touchant un point d'arrêt.
- **Effet automatique** — 4 points d'arrêt obligatoires :
 1. Décision archi irréversible (BDD, contrat d'API, suppression de service).
 2. Ambiguïté métier non résoluble (l'agent ne doit pas inventer).
 3. Conflit avec une contrainte explicite (perf, sécurité, budget).
 4. Échec reproductible après 3 tentatives → escalade.

- Garde-fous : branche dédiée jamais sur **main** , checkpoint Git avant chaque modif, alerte budget tokens.
- Niveau d'autonomie **modulé par le blast radius** (local = plus autonome).
- **Artefact** : règles dans **AGENTS.md** , **scripts/guard.sh** (blocage après N tentatives). Permissions dans **opencode.json** .

Point de vigilance ouvert : sous OpenCode, **@name** ne garantit pas le confinement des permissions (constat session craftech-app). À tester avant de compter dessus (voir session-log).

4. Tiering de modèles **[OPÉRATIONNEL]**

L'allocation de puissance. Le bon modèle pour la bonne tâche, ni gaspillage ni sous-qualité.

- **Déclencheur** : à l'entrée de chaque phase / type de tâche.
- **Application** : chaque phase du cycle long ; routage décidé par phase **et** blast radius.
- **Effet automatique** — matrice phase × profil :
 - Exploration/clarification → rapide, faible coût.
 - Documentation/spec → équilibré.
 - Génération de code → haute qualité.
 - Validation/review → puissant, **différent du codeur**.
 - Orchestration → léger, bon en tool-use.
 - Le modèle utilisé est tracé dans le CDLC (métriques).
- **Artefact** : modèle par phase spécifié dans **AGENTS.md** ; assistants distincts dans Cherry Studio ; script routeur (LiteLLM) en cible.

5. CDLC — contexte comme entité vivante **[PARTIEL]**

Le substrat. Tout le reste lit et écrit dedans. Niveau 1 opérationnel, niveaux 2-3 à activer progressivement.

- { **Déclencheur** : permanent. Quatre opérations continues sur le contexte.
- { **Application** : englobe les deux cycles ; point d'entrée unique via le Context Hub (strate 0, chargé en premier).
- { **Effet automatique** — les 4 opérations :
 - { **Generate** → rendre explicite l'implicite (**docs/** , **context/** , **AGENTS.md**).
 - { **Evaluate** → TDD du contexte, evals avant la spec (**check-context-hub.sh**).
 - { **Distribute** → versionner le contexte comme une dépendance (Git, SemVer).
 - { **Observe** → capturer les hésitations de l'agent (**meta/** , **context.db**).
 - { Règle des deux explications : une contrainte expliquée 2× en 2 sessions = bug de doc → MAJ la spec, pas le chat.
- { **Artefact** : **AGENTS.md** + **docs/** versionnés **[OPÉRATIONNEL]** · **context.db** SQLite (faits + statut + horodatage) **[DIFFÉRÉ]** · KB Cherry Studio (sémantique) **[DIFFÉRÉ]** .

6. Mémoire des échecs **[DIFFÉRÉ]**

Le corpus bug-solution. Transforme chaque bug résolu en diagnostic instantané la fois suivante. À amorcer dès les premiers bugs non triviaux de Garmin.

- { **Déclencheur** : avant tout diagnostic (grep du symptôme) et après toute résolution non triviale (documentation).
- { **Application** : phases Développement/Test et Maintenance du cycle long.
- { **Effet automatique** :
 - { Avant diagnostic → **grep** du message d'erreur dans le corpus ; si trouvé, solution documentée présentée.
 - { Après résolution → l'agent documente (symptôme → diagnostic → solution).
 - { Pattern agrégé dès **3 occurrences** ; 3 occurrences d'un même bug → escalade (lien contrat d'autonomie).
 - { Un bug **critical** augmente le niveau de risque futur (lien blast radius).
- { **Artefact** : **context/bugs/** (un sous-dossier par module), **TEMPLATE.md** , **patterns/common-solutions.md** . Amorçage : documenter les 5 derniers bugs résolus manuellement.

7. Méta-boucle [DIFFÉRÉ]

L'auto-amélioration. La méthode apprend de son propre usage. L'enveloppe : elle agit non sur le code mais sur les artefacts de la méthode eux-mêmes.

- **Déclencheur** : note rapide par feature (scratchpad) ; synthèse en fin de sprint.
- **Application** : englobe tout — reboucle de la fin vers les artefacts de départ (skills, contrat d'autonomie, tiering, blast radius).
- **Effet automatique** :
 1. Note par feature. 2. En fin de sprint, l'agent génère une ébauche de rétrospective. 3. L'humain valide et **transforme les constats en modifications d'artefacts**. 4. Commit de la rétro + des changements liés.
 - Exemple : un sprint révèle trop de corrections de typage → modifier le skill **implement** pour lancer le vérificateur de types avant de proposer le code.
- **Artefact** : **context/meta/retrospectives.md** · **patterns.md** · **metrics.md** .
Ce **session-log.md** en est déjà l'amorce manuelle.

Carte des dépendances entre systèmes

Ce que le diagramme montre spatialement, résumé en règles :

- Le **blast radius** règle → quality gates, contrat d'autonomie, tiering.
- Les **quality gates** alimentent → CDLC (échec = Observe→Generate).
- La **mémoire des échecs** alimente → contrat d'autonomie (3 occurrences = escalade) et blast radius (bug critique = risque accru).
- Le **CDLC** contient les traces de tous les autres.
- La **méta-boucle** modifie tous les autres.

Ordre d'activation recommandé (pour Garmin)

1. Blast radius + quality gates + contrat d'autonomie + tiering → dès le départ (opérationnels, ce sont le quotidien).

2. { CDLC niveau 1 (**AGENTS.md** + **docs/**) → dès le départ ; niveaux 2-3 quand le volume de contexte fait mal.
3. { Mémoire des échecs → au premier bug non trivial rencontré.
4. { Méta-boucle → à la première rétrospective de fin de sprint.